ASSESSMENT COMPANION

Quiz answer key

600 answers with explanations

Attempt the questions before opening this key. These answers guide self-correction; acceptable reasoning may use different wording. A passing short quiz does not replace the lab, assigned exercises, project review or graduate research work.

200 lesson quizzes / 3 questions each / Entry diagnostic

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Module 01: Starting as an engineer

01.1 What a program actually does

1. State is the underlying information; the image is one view of that information.

2. 2 + 3 - 1 + 0 - 4 + 2 = 2.

3. Rendering may occur at a different rate or multiple times; movement must follow simulation rules, not draw count.

01.2 Files, terminals and reproducible execution

1. The process working directory, unless the application explicitly resolves it another way.

2. No. It is normally a shell command; Python has separate filesystem APIs.

3. The exact source revision/file, tool environment and command; inputs and working directory also matter.

01.3 Trace before running

1. 3. The earlier integer value was bound to b; changing a does not recompute b.

2. The first assignment overwrites the old x, so both names end with the old y.

3. It exposes the specific mistaken assumption, which can then be tested instead of hidden by random edits.

01.4 Units, budgets and engineering notebooks

1. 1 unit, because 8 multiplied by 0.125 is 1.

2. 1000/120 = approximately 8.33 milliseconds.

3. The result depends on the environment and work performed; without them it cannot be meaningfully reproduced or generalized.

Module 02: Values, decisions and repetition

02.1 Types and expressions

1. One is text and one is an integer; their operations and validation rules differ.

2. The string "abcd".

3. No. An integer directly represents whole counts and avoids fractional or approximation ambiguity.

02.2 Conditions and boundary cases

1. No. Use >= when exact funds are sufficient.

2. False: the parenthesized conjunction is false, and neither side of or is true.

3. Those cases expose incorrect inclusive/exclusive comparisons and missing boundary rules.

02.3 Loops and invariants

1. Five: 2, 3, 4, 5 and 6.

2. The body must eventually reduce remaining or otherwise make the condition false, unless it exits explicitly.

3. At the chosen loop boundary, the item count is always between zero and ten inclusive.

02.4 Functions and contracts

1. 4, because 2 + 8 times 0.25 equals 4.

2. Other code can consume and test the result without parsing console output.

3. Valid ranges/types, units, outputs, side effects and failure behavior.

Module 03: Organizing data and small programs

03.1 Sequences, dictionaries and identity

1. When the main operation is association or lookup by a stable key rather than by position.

2. No. Both names refer to the same list object.

3. Its nested objects can remain the same mutable objects referenced by the original.

03.2 Decomposition, modules and interfaces

1. The supported operations and contracts through which another part of the system interacts with a component.

2. Imports should make reusable definitions available; automatic interaction prevents tests and other tools from using them cleanly.

3. Recorded commands can be fed back through the same simulation interface.

03.3 Files, serialization and validation

1. No. Syntax validity does not establish schema, range or game-rule validity.

2. That the intended data survives serialization and deserialization without unacceptable change.

3. It lets the loader detect compatibility requirements and choose migration or a clear rejection.

03.4 Classes, composition and state machines

1. An object's invariants by controlling how its state can be changed.

2. They allow combinations that may not correspond to any valid behavior unless separately constrained.

3. When independent capabilities should be combined without forcing them into one rigid inheritance tree.

Module 04: Debugging, testing and collaboration

04.1 Debugging as an experiment

1. Reproduction steps, inputs/environment, expected behavior, actual behavior and useful evidence.

2. The final crash may be a downstream symptom; the first deviation is closer to the cause.

3. A test that preserves a previously failing case to detect reintroduction of that defect.

04.2 Unit, integration and property tests

1. It checks cooperation between components rather than one isolated responsibility.

2. The test can reproduce the same mistake and falsely agree with the implementation.

3. The exercised checks passed in that environment; it does not prove every requirement or input is correct.

04.3 Git, branches and recoverable changes

1. Which current file changes become part of the next commit.

2. Earlier commits may still retain them and copies may already exist.

3. It identifies the behavior or purpose of the change and, when needed, its reasoning.

04.4 Review, documentation and responsible AI help

1. The change, its behavior and evidence, not the author's character.

2. Compilation checks only some structural rules; logic, security, performance and requirements can still be wrong.

3. Explaining the reasoning, testing edge cases and solving a related new task independently.

Module 05: A complete first game

05.1 The update loop and command model

1. No. Simulation and presentation can run at different rates.

2. The simulation state should remain unchanged.

3. It enables automated tests, replays and server-style execution without a graphical interface.

05.2 Collision rules and complete behavior

1. Boundary, obstacle, terminal-state and invalid-command policies, among other possible rules.

2. The right move is rejected and x stays 9.

3. It avoids leaving the system partially updated when one requirement fails.

05.3 Deterministic replay and simple measurement

1. Initial state, order/tick placement, relevant configuration and controlled randomness/environment assumptions.

2. Otherwise the measurement may primarily reflect terminal I/O instead of simulation work.

3. No. It is evidence for that case; broader tests and analysis are still required.

05.4 Finishing, usability and the first portfolio artifact

1. Prerequisites, exact run/test commands, controls, expected behavior and known limitations.

2. Some players cannot reliably distinguish it; a second channel such as text or shape preserves the information.

3. A reproducible comparison with the same workload and clearly stated measurement conditions.

Module 06: Discrete mathematical reasoning

06.1 Algebra, functions, logarithms and trigonometry

1. 6, because 2^6 = 64.

2. pi radians.

3. Time cannot be zero; also state the physical units and whether negative time is meaningful.

06.2 Logic, sets and quantified claims

1. When P is true and Q is false.

2. There exists a unit whose health is not positive.

3. The claim can change meaning or truth when it ranges over different objects or inputs.

06.3 Proofs, induction and invariants

1. A valid base case and an implication from the inductive hypothesis to the next case.

2. 100*101/2 = 5050.

3. The code, machine arithmetic or environmental assumptions may differ from the proven model.

06.4 Counting, graphs and recurrences

1. 2^5 = 32.

2. 5*4/2 = 10.

3. No member can be initialized first while requiring another member of the cycle to be ready.

Module 07: Linear algebra and game geometry

07.1 Vectors, dot products and projections

1. 10.

2. 0, indicating perpendicular directions.

3. Its length is zero, so division is undefined and it has no unique direction.

07.2 Matrices, bases and linear systems

1. No. Transformation order usually changes the result.

2. Information on that axis is lost, making the transformation singular.

3. Factorization can be more efficient and numerically preferable, especially for repeated or large solves.

07.3 Homogeneous transforms, cameras and quaternions

1. Translation should not change a pure direction; the homogeneous form makes the translation term vanish.

2. It expresses world-space geometry in the camera coordinate system.

3. The library convention and application convention must agree; otherwise valid arithmetic produces the wrong geometry.

07.4 Eigenvalues, least squares and singular values

1. 3 and 2, with the coordinate axes as eigenvector directions.

2. The sum of squared residuals under its specified model.

3. No. It can limit added numerical error but cannot eliminate the problem's inherent sensitivity.

Module 08: Calculus, mechanics and numerical simulation

08.1 Limits, derivatives and rates

1. dx/dt = 6*t.

2. Meters per second squared.

3. Subtracting nearly equal floating-point values magnifies rounding and cancellation error.

08.2 Integrals, accumulation and continuous probability

1. Displacement is 10 units of length.

2. [t^2] from 0 to 3 = 9.

3. Positive and negative displacement can cancel; total distance is a different quantity.

08.3 Newtonian mechanics and differential equations

1. 5 m/s^2.

2. Impulse is force integrated over time and changes momentum; force is a rate of momentum change.

3. Many trajectories can satisfy the same differential rule; initial state selects the intended one.

08.4 Time integration, stability and error

1. The newly updated velocity.

2. No. Stability still depends on the equations, method and step size.

3. Compare to an analytic solution or conserved quantity and perform timestep-refinement tests.

Module 09: Probability, statistics and evidence

09.1 Probability and conditional reasoning

1. 0.1, the product under the stated independence assumption.

2. No. They condition on different populations.

3. Even a small false-positive rate can produce many alerts when true events are very rare.

09.2 Random variables, distributions and sampling

1. 5.

2. No. It describes an average under the model, not an individual outcome.

3. An extra draw in one system should not unintentionally alter another system's sequence.

09.3 Inference, uncertainty and benchmark design

1. They share workload phases, caches, thermal state and other correlated conditions.

2. No. Statistical evidence and practical effect size are different questions.

3. It exposes slow cases that can affect responsiveness despite an acceptable average.

09.4 Regression, correlation and ethical telemetry

1. A model can overfit and fail on new observations or workloads.

2. A variable that influences both the proposed explanatory factor and the outcome, complicating causal interpretation.

3. It reduces privacy exposure and unnecessary data handling while keeping the experiment focused.

Module 10: Signals, optimization and numerical foundations

10.1 Multivariable calculus and gradients

1. (2*x, 6*y).

2. The first-order local mapping from small input changes to output changes for a vector-valued function.

3. No. It may identify a maximum, saddle or other stationary point.

10.2 Sampling, aliasing and Fourier ideas

1. 24,000 Hz, the half-sampling-rate boundary; practical filtering requires additional care.

2. To remove frequencies that would fold into misleading lower frequencies at the new sample rate.

3. No. Distinct original signals may already have become indistinguishable in the samples.

10.3 Convolution, filters and systems

1. The filter's impulse response, including its alignment convention.

2. 5.

3. It changes the mathematical result at the edges and may create visible or audible artifacts.

10.4 Optimization, conditioning and approximation budgets

1. At x=3, with objective value zero.

2. It can prevent updates from overshooting the region where local information is useful, though convergence can become slower.

3. Report nonconvergence or the achieved residual explicitly rather than pretending the requested accuracy was met.

Module 11: Native programming with C++

11.1 Compilation, linking and build systems

1. It resolves symbol references and combines object code/libraries into the requested output.

2. No. A referenced function generally needs a matching definition unless special language rules apply.

3. Optimization, instrumentation and assertions can materially change behavior and performance.

11.2 Integer, floating-point and object representations

1. 256.

2. No. It is undefined behavior; use checked arithmetic or an intentionally suitable representation.

3. Padding, byte order, alignment, type widths and representation may vary.

11.3 Values, references, pointers and lifetimes

1. No. Ownership must be established by the interface or an owning type.

2. The local object is destroyed at function exit, leaving a dangling reference.

3. No. Thread safety depends on all accesses, object behavior and synchronization.

11.4 Standard containers, iterators and error handling

1. No. It changes capacity, not the number of constructed elements.

2. Reallocation may move elements to a different storage region.

3. It hides failures and may allow incorrect or partially updated state to appear successful.

Module 12: Ownership, abstractions and C++ design

12.1 RAII and resource ownership

1. Resource ownership and release to the lifetime of an object.

2. Yes, if owning references keep each other alive; weak/non-owning links may be needed.

3. A release operation may require a live device or context; destroying the device first invalidates that assumption.

12.2 Copy, move and exception safety

1. No. It permits move-aware overload selection; the selected operation determines what happens.

2. On failure, the operation leaves the relevant observable state unchanged.

3. Correct member types can manage resources without fragile custom copy/move/destruction code.

12.3 Interfaces, composition and polymorphism

1. An implementation used through an interface must preserve that interface's behavioral contract.

2. Deletion through a base pointer must invoke appropriate derived cleanup.

3. It increases coupling, obscures contracts and makes isolated testing or replacement difficult.

12.4 Templates, concepts and data-oriented choices

1. The compile-time requirements a type or expression must satisfy for a template interface.

2. No. Its benefit depends on access pattern, workload and architecture.

3. Compilation time, error complexity, binary size or maintenance burden.

Module 13: Data structures and their costs

13.1 Complexity, arrays and linked structures

1. No. It bounds growth under a cost model and omits constants and environment details.

2. n squared, since the sum is n(n-1)/2.

3. It bounds total cost over an operation sequence without requiring a probability distribution over inputs.

13.2 Stacks, queues and ring buffers

1. B.

2. Equal index values can otherwise be ambiguous after wrapping.

3. No. Concurrent access introduces ordering, atomicity and ownership requirements.

13.3 Hash tables and collision policies

1. No. Collisions are possible; equality must still be checked.

2. 3 entries per bucket on average for this definition.

3. It depends on hashing, occupancy and input assumptions; worst cases can be much slower.

13.4 Trees, heaps and priority queues

1. No. It guarantees the heap ordering relation, especially access to the extreme element.

2. n in the chain-like worst case, depending on height convention by one.

3. Equal priorities otherwise may produce inconsistent processing order, complicating tests and deterministic behavior.

Module 14: Algorithms for maps, simulation and tools

14.1 Searching, sorting and algorithm selection

1. A range ordered consistently with its comparison rule.

2. The original relative order of elements considered equivalent by the comparison.

3. An output can be sorted yet incorrectly omit, duplicate or change input elements.

14.2 Graph traversal and dependency ordering

1. On an unweighted graph, or when all traversed edges have equal cost.

2. No.

3. To avoid repeated exploration and nontermination on cycles while preserving the traversal's intended behavior.

14.3 Dijkstra, A* and admissible heuristics

1. Nonnegative edge costs.

2. 7.

3. A better path to a previously processed node may be discovered later; permanently closing it can discard that improvement.

14.4 Dynamic programming, complexity limits and approximation

1. Overlapping subproblems and a valid way to combine their solutions through sufficient state.

2. No. It concerns worst-case problem-family complexity, not every individual instance.

3. It provides a simple correctness oracle for testing more sophisticated algorithms.

Module 15: Programming languages, compilers and computability

15.1 Language paradigms and type systems

1. No. It enforces its type rules, which cover only part of correctness.

2. The same explicit inputs determine the result without hidden state or side effects.

3. Who allocates and frees resources, valid lifetimes, error signaling and compatible representations.

15.2 Lexing, parsing and syntax trees

1. Lexing identifies tokens; parsing organizes tokens according to grammar.

2. 14.

3. They allow later semantic or runtime errors to point back to meaningful user code.

15.3 Bytecode, virtual machines and bounded execution

1. Minus one: two values are removed and one is added.

2. Independent execution paths provide a useful differential check on compilation and interpretation.

3. No. Host capabilities, memory safety, limits and implementation flaws still determine isolation.

15.4 Automata, computability and compiler optimization

1. No. Many restricted or specific programs admit termination proofs.

2. Rounding occurs after operations, so changing grouping can change which information is lost.

3. It must preserve the required observable behavior under the language and application assumptions.

Module 16: Processor architecture and assembly

16.1 Instruction sets, registers and calling conventions

1. The ISA is the software-visible contract; the microarchitecture is a particular implementation.

2. Argument/return placement, register preservation and related function-call rules.

3. Unsupported instructions or extensions cannot be assumed to behave like a complete implementation.

16.2 Pipelines, dependencies and branch prediction

1. An operation requires a result produced by an earlier operation.

2. No. Work per cycle, stalls, memory and workload behavior also matter.

3. They can expose parallel operations instead of one serial dependency chain, subject to resource and numerical constraints.

16.3 Caches, locality and memory hierarchy

1. Accessing data near data that was recently accessed.

2. 16, ignoring boundary misalignment and other storage.

3. No. Another cache level or other hierarchy component may satisfy it.

16.4 Virtual memory, SIMD and architectural limits

1. Virtual-to-physical address translation information and associated access details.

2. No. Memory, dependencies, overhead and other resources can limit improvement.

3. The remainder must be handled without out-of-bounds accesses or omitted elements.

Module 17: Operating systems and resource management

17.1 Processes, threads, system calls and protection

1. The process address space and many resources, while retaining separate execution state.

2. A controlled request from application code for an operating-system kernel service.

3. Stronger isolation can contain faults, though communication and resource costs may be higher.

17.2 Allocation, paging and memory-mapped data

1. No.

2. The allocator or runtime may retain storage for later reuse rather than returning it immediately to the OS.

3. No. Access can still trigger page faults and underlying I/O.

17.3 Scheduling, responsiveness and real-time constraints

1. Higher-priority work is delayed by lower-priority work through a shared-resource dependency.

2. No. Bursts, blocking and scheduling delays can still cause misses.

3. Its unpredictable delay can exceed the callback deadline and cause audible interruption.

17.4 Filesystems, persistence and asynchronous I/O

1. No. Data may remain buffered unless the required durability operations and guarantees are satisfied.

2. The operation may still read or write that storage after the initiating call returns.

3. Partial writes, malformed data, interruption points and replacement failures under an explicit failure model.

Module 18: Concurrency and parallel programming

18.1 Races, mutexes and happens-before

1. Yes. A multi-step invariant can be violated by interleaving even when each access is atomic.

2. The entire shared invariant and every conflicting access that participates in it.

3. Scheduling is not the required synchronization guarantee and can change across runs or systems.

18.2 Condition variables, deadlocks and cancellation

1. Wakeups can be spurious or another thread can consume the condition before the waiter proceeds.

2. A common source of circular wait and deadlock.

3. Without new work or a notification, they may never observe the stop condition.

18.3 Atomics, memory ordering and false sharing

1. No.

2. Coherence contention caused by independent data sharing a cache line.

3. Another thread may still hold or access the node; safe reclamation needs a separate protocol.

18.4 Task graphs, work stealing and scaling limits

1. 4x, since the parallel portion cannot eliminate the remaining 0.25 of runtime.

2. Scheduling, synchronization and bookkeeping can cost more than the useful work.

3. A required ordering or data-availability relationship before a dependent task may execute.

Module 19: Networks, storage and data services

19.1 Network layers, latency and transport semantics

1. No. It supplies an ordered byte stream.

2. No. Capacity and delay are different properties.

3. Whatever reliability, ordering, congestion control, validation and security its requirements need.

19.2 Protocol design, serialization and timeouts

1. No. The operation may have executed while the response was delayed or lost.

2. An attacker or malformed file could request excessive memory or trigger overflow.

3. Repeating an operation has the same intended effect as applying it once.

19.3 Relational modeling, SQL and indexes

1. A specified relationship by requiring referenced values to exist, subject to its defined rules.

2. The index structures must be maintained alongside the table changes.

3. It keeps input data separate from SQL structure and prevents common injection mistakes.

19.4 Transactions, consistency and distributed failure

1. The transaction's changes are treated as all-or-nothing under the database guarantee.

2. No. Cross-service coordination needs a separate protocol.

3. Total item quantity, valid nonnegative balances and the intended ownership changes.

Module 20: Engineering practice, security and maintainability

20.1 Requirements, architecture and design records

1. Observable behavior, conditions and an explicit acceptance criterion.

2. They preserve the reasoning and make future reassessment more informed.

3. No. The implemented result must still meet the specified needs and constraints.

20.2 Defensive programming and threat modeling

1. Authentication concerns identity; authorization concerns allowed actions.

2. The player controls that environment and can send incorrect or manipulated requests.

3. No. Authenticated or encrypted input can still be malformed, malicious or contrary to game rules.

20.3 Build automation, dependencies and release discipline

1. To reduce uncontrolled version changes and support reproducible builds.

2. Its source revision, build configuration, toolchain and dependency/asset versions.

3. To verify it actually blocks bad changes rather than merely displaying success regardless of checks.

20.4 Ethics, accessibility, teamwork and sustainable delivery

1. No. Copyright and license terms still govern use.

2. Otherwise a technically functioning feature may remain unusable for part of the intended audience.

3. Contributing system/process conditions and practical improvements rather than personal blame.

Module 21: From geometry to a software renderer

21.1 Images, color and coordinate conventions

1. Rows may contain padding or use a layout different from width times bytes per pixel.

2. 320 x 180 x 4 = 230,400 bytes.

3. No. It represents non-color data, so that transformation would corrupt its intended values.

21.2 Projection and clipping before perspective division

1. Clipping in homogeneous space handles view-volume boundaries and problematic w values before division.

2. 0.5.

3. No. New intersection vertices are generally needed to preserve the visible polygon.

21.3 Triangle coverage, depth and perspective-correct interpolation

1. One for the normalized representation of a point relative to a nondegenerate triangle.

2. When all relevant clip w values are equal.

3. To assign boundary samples consistently and avoid cracks or inconsistent double ownership.

21.4 Rays, intersections and acceleration structures

1. When direction has unit length and the coordinate units represent that distance.

2. (0,0,6).

3. It provides a simple reference for testing the accelerated implementation and measuring whether acceleration helps.

Module 22: Explicit GPU programming and graphics APIs

22.1 GPU work submission, resources and pipeline state

1. No. Work normally executes asynchronously.

2. Submitted GPU commands may still reference it.

3. No. One capability-compatible backend plus clear interfaces is the intended first step.

22.2 Buffers, shaders, layouts and resource binding

1. The host and shader can apply different padding, alignment or matrix-layout rules.

2. 16, with bounds checks for 24 excess invocations.

3. Which resource a shader interface can access, under the API and lifetime contract.

22.3 Synchronization, hazards and frames in flight

1. No. Correct synchronization must satisfy the relevant requirements for both.

2. After the work that references its resources has completed according to the API contract.

3. It can conceal missing dependencies while eliminating useful overlap and increasing overhead.

22.4 Presentation, resizing and device capability policy

1. Depth images, presentation images and other size-dependent resources may also need safe recreation.

2. Skip or suspend dependent rendering and avoid invalid allocation or division.

3. To distinguish requirements, optional enhancements and fallbacks, making compatibility predictable.

Module 23: Lighting, materials and real-time image quality

23.1 Radiometry, reflectance and physically based materials

1. Incoming illumination direction and outgoing reflected radiance direction at a surface, under its formal definition.

2. rho/pi.

3. The numerical value depends on the model and parameter mapping, so incompatible encodings change appearance.

23.2 Textures, mipmaps, filtering and tangent-space detail

1. Efficient prefiltering for minification, reducing aliasing when many texels map to a pixel.

2. Four thirds of the base level for a full large two-dimensional power-of-two chain.

3. They can change tangent-frame handedness unless the basis and encoding account for it.

23.3 Shadows, light selection and rendering-path tradeoffs

1. It can detach the apparent shadow from the casting or receiving surface.

2. No. Writing and reading intermediate surface buffers can consume substantial bandwidth.

3. Scene light distribution, overlap, setup cost, shading work and the target hardware, measured under a defined workload.

23.4 HDR, tone mapping, temporal reconstruction and validation

1. No. Tone mapping maps rendered values toward a display representation.

2. The previous frame generally no longer corresponds to the current visibility and coordinates.

3. Ghosting, instability and delayed response are properties of sequences, not only isolated frames.

Module 24: Engine architecture, simulation and resource ownership

24.1 Fixed updates, interpolation and overload policy

1. 5/10 = 0.5.

2. No. Other ordering, input and numerical differences can still change results.

3. An overloaded system can spend all its time catching up and fall further behind.

24.2 Entities, components, handles and data-oriented storage

1. Objects can move or storage can be reused while logical identity must remain controlled.

2. No.

3. Immediate changes can invalidate iterators or alter which elements are visited.

24.3 Allocators, task graphs and frame-resource lifetime

1. When no live operation or object still relies on its contents.

2. No. The dependency and visibility requirements must be satisfied.

3. It provides a simpler correctness baseline before scheduling and synchronization add complexity.

24.4 Subsystem boundaries, save formats and reloadability

1. The address has no stable meaning in a later process and may expose unsafe or invalid state.

2. An explicit compatibility decision, migration path or clear rejection.

3. Shutdown must safely release only resources that were successfully acquired.

Module 25: Asset pipelines, editors and an integrated engine

25.1 Importing, cooking and content-addressed asset builds

1. Their editing, storage, validation and loading requirements differ.

2. Yes, because it changes the transformation result.

3. A tool change can alter output even when source content is unchanged.

25.2 Streaming, residency and memory budgets

1. Decoded, staging and final resources may be larger and coexist.

2. Use a defined fallback or deferred behavior rather than dereferencing missing data.

3. It makes transitions, failures, cancellation and ownership testable.

25.3 Editors, inspection and undoable commands

1. It separates underlying document state from the interface used to display and edit it.

2. The target identity and both the old and new transforms.

3. Otherwise running or stopping a simulation can unpredictably overwrite the intended scene.

25.4 Engine integration and reproducible demonstration

1. 1/60 second, approximately 16.67 ms.

2. Averages can conceal stalls and unstable responsiveness.

3. Missing dependencies, undeclared assets, environment assumptions and incomplete instructions.

Module 26: Collision detection and physical simulation

26.1 Broad phase, narrow phase and geometric robustness

1. Yes. Narrow-phase tests can reject them; false negatives are the dangerous case.

2. 4,950.

3. Impulse and separation calculations depend on a consistent direction between the two bodies.

26.2 Rigid bodies, angular motion and collision impulses

1. 3 m/s in the impulse direction.

2. It changes angular momentum through the lever arm relative to the center of mass.

3. It naturally produces no velocity change under impulse calculations without relying on infinite numerical values.

26.3 Constraints, iterative solvers and continuous collision detection

1. 0.5 m.

2. It can reduce residual constraint error, though cost rises and other modeling problems can remain.

3. No. It addresses certain between-step collision failures under its motion and solver assumptions.

26.4 Physics testing, determinism and engine integration

1. No. Compiler, arithmetic and implementation differences can still matter.

2. It localizes the origin of disagreement before later effects amplify it.

3. Capture a useful diagnostic and reject or isolate invalid state according to policy rather than letting it propagate silently.

Module 27: Animation, audio and interactive timing

27.1 Skeletons, transforms and skinning

1. A child world transform depends on the parent world transform.

2. One.

3. The result may cease to be a valid pure rotation and can introduce scale or shear.

27.2 Animation blending, state machines and inverse kinematics

1. No. Geometry, joint limits and constraints can make targets unreachable.

2. 2 units.

3. Animation and physical simulation otherwise may apply incompatible displacement to the same object.

27.3 Digital audio, mixing and real-time callback discipline

1. Approximately 5.33 ms.

2. Their unpredictable blocking time can exceed the audio deadline.

3. No. It can reduce buffering latency while increasing deadline pressure and underrun risk.

27.4 Spatial audio, animation events and multimodal accessibility

1. No. It represents only some directional cues.

2. To recognize previously presented events and apply a deliberate deduplication policy.

3. The player may not hear the cue because of disability, environment or settings.

Module 28: Game AI, decision systems and machine learning

28.1 Navigation graphs, steering and dynamic obstacles

1. Pathfinding selects a route; steering produces local movement to follow or adjust that route.

2. Not under the stated no-overlap model.

3. Large or frequent searches can consume the frame budget and harm responsiveness.

28.2 Behavior trees, utility systems and explainable decisions

1. At least a defined running/progress state, success, failure and cancellation behavior.

2. Small input changes repeatedly reverse the preferred action.

3. It makes selection criteria and failures visible for testing, tuning and designer review.

28.3 Adversarial search, planning and simulation budgets

1. No. It prunes branches that cannot affect the same result under the search assumptions.

2. About 10^6, or one million.

3. The decision deadline may arrive before the planned search completes.

28.4 Learning systems, neural inference and responsible evaluation

1. It reduces leakage from highly correlated examples belonging to the same episode.

2. The chain rule applied through the computation graph.

3. It reveals whether learning adds useful quality relative to its data, runtime and maintenance costs.

Module 29: Multiplayer simulation and online services

29.1 Authority, prediction and reconciliation

1. The state accepted as the deciding game truth under the chosen trust model.

2. Unacknowledged inputs 41 through 44.

3. The authority can validate and compute outcomes instead of trusting client-supplied benefits.

29.2 Snapshots, interpolation, relevance and lag compensation

1. Additional presentation delay from buffering earlier snapshots.

2. The receiver needs the correct prior state to reconstruct the delta.

3. A modified client may inspect received information even if the normal UI hides it.

29.3 Lockstep, rollback and deterministic replay

1. Two seconds.

2. 240 MB.

3. Re-executing a tick must not repeat irreversible actions such as issuing a reward or sending an external message.

29.4 Match services, identity, persistence and failure recovery

1. No. Identity verification and permission to perform a specific action are separate checks.

2. Retries can then be recognized and processed without duplicating its durable effect.

3. No. Local services and synthetic accounts satisfy the educational objective.

Module 30: Production quality and the undergraduate engine capstone

30.1 Profiling, frame budgets and performance regression tests

1. 18 ms unchanged work plus 0.2 ms optimized work = 18.2 ms.

2. Debug and optimized builds can have very different performance and checks.

3. That operation may not dominate the real critical path or may create costs elsewhere.

30.2 Compatibility, accessibility, localization and human testing

1. It exposes layout and text-handling problems using expanded or altered test strings.

2. The current binding, not an obsolete hard-coded default.

3. No. It supports engineering review but is not a legal determination or substitute for user evaluation.

30.3 Packaging, patching, modding and supply-chain boundaries

1. Yes. Malformed sizes, structures or resource demands can trigger failures unless validated.

2. Updates or reinstalls should not erase user progress.

3. No. Isolation depends on the enforced capability, memory and resource boundaries.

30.4 Capstone review, release and technical postmortem

1. A reproducible package, tests, usable controls, documented limitations and independent execution.

2. It distinguishes the learner's decisions and understanding from supplied work and supports provenance.

3. It tests whether the learner can reason about and change the system rather than only present a prepared artifact.

Module 31: Circuits, measurement and electronic foundations

31.1 Voltage, current, resistance and safe measurement

1. 0.005 A, or 5 mA.

2. 5 V x 0.005 A = 0.025 W, or 25 mW.

3. It can create an unintended low-resistance path or short, damaging equipment or causing injury.

31.2 Capacitors, inductors, transients and frequency response

1. 10 ms.

2. Approximately 1-exp(-1), or 63.2%.

3. Energy already stored in capacitors or inductors affects the transient response.

31.3 Diodes, transistors, CMOS and analog interfaces

1. Its electrical behavior is continuous and depends on voltages, currents, device properties and operating region.

2. No. Electrical input limits and output levels must also match.

3. Absolute limits are not a normal design target or a guarantee of intended performance.

31.4 Sampling, ADCs, DACs and noisy controller inputs

1. 4,096.

2. No. Noise and other errors can reduce accuracy and effective resolution.

3. Once high-frequency content aliases into the sampled band, ordinary later filtering cannot distinguish it from genuine in-band content.

Module 32: Digital logic, state and timing

32.1 Boolean logic and combinational building blocks

1. Combinational outputs depend on current inputs without stored state; sequential behavior also depends on state.

2. Selects one of two inputs according to a control bit.

3. No. Physical mapping, fanout and timing also affect the implementation.

32.2 Registers, finite-state machines and synchronous timing

1. The value presented under the specified timing conditions at its active clock event.

2. 125 MHz before additional uncertainty or margin.

3. No. Hold concerns the minimum-delay relationship near the capture edge.

32.3 Binary arithmetic, fixed point and overflow policies

1. -128 through 127.

2. 1.5.

3. The product can require more bits and has a changed fixed-point scale before rounding or rescaling.

32.4 Clock domains, reset and metastability

1. No. It reduces propagation probability under stated timing and device assumptions.

2. Bits can be observed from different source transitions, yielding an incoherent word.

3. No. It does not faithfully model that analog probability and timing behavior.

Module 33: SystemVerilog and verification-driven RTL design

33.1 RTL semantics and synthesizable SystemVerilog

1. Missing assignments can imply unwanted retained state or latches.

2. Each register receives the other register's previous value.

3. No. It may elaborate into replicated combinational or parallel hardware.

33.2 Testbenches, scoreboards and constrained stimulus

1. Accepted transactions according to the interface handshake.

2. They allow a failing stimulus sequence to be reproduced.

3. It checks that the verification environment can detect the failure it claims to cover.

33.3 Assertions, coverage and formal reasoning

1. No. The bins may omit important behavior and coverage does not establish every property.

2. They can exclude the very input sequences that trigger it.

3. The checked time or state bound, unless an additional argument proves the property unboundedly.

33.4 Synthesis, constraints and FPGA resource mapping

1. No. Implementation and timing analysis under correct constraints are additional requirements.

2. The source description may map to registers or memories with different cost and behavior.

3. Otherwise reported area or timing differences may reflect the experiment setup rather than the design change.

Module 34: Designing a small processor

34.1 Instruction sets, datapaths and single-cycle execution

1. The ISA specifies programmer-visible behavior; the microarchitecture is one implementation of it.

2. The longest required combinational path plus sequential timing and margin.

3. Unsupported instructions or required architectural behaviors may prevent compliance.

34.2 Pipelining, forwarding, stalls and branch recovery

1. No. Its main goal is often improved throughput by overlapping instructions.

2. They are not part of the accepted instruction path and must not alter architectural state.

3. A following instruction needs loaded data before the pipeline has made it available.

34.3 Memory interfaces, caches and protection boundaries

1. It omits latency, backpressure, errors and device-specific side effects.

2. Which memory block is represented by the stored cache line.

3. No, but omitting translation and protection changes its capabilities and safe use cases.

34.4 Processor verification, toolchains and architectural evidence

1. Different correct implementations can have different internal timing while preserving the same architectural effects.

2. The compiler or runtime may use operations unsupported by the educational core.

3. Program bytes, initial state, toolchain/configuration, seed and the first architectural mismatch.

Module 35: FPGA-based game subsystems and integration

35.1 Memory-mapped I/O, timers, interrupts and DMA

1. Accesses can have side effects, ordering requirements and restricted widths.

2. After the device has completed the access and ownership is returned under the protocol.

3. No. It is not a substitute for synchronization or the platform's device-access rules.

35.2 Framebuffers, scanout and pixel pipelines

1. 320 x 240 x 2 = 153,600 bytes.

2. No. Blanking, encoding, protocol and other traffic can add overhead.

3. An arbitrary swap can display parts of different frames, causing tearing.

35.3 A small accelerator and its software contract

1. 80 + 10 = 90 microseconds.

2. It prevents a producer from overrunning a consumer that cannot currently accept more data.

3. A pipeline can accept frequent work while each individual item still takes multiple cycles to complete.

35.4 Integrated digital-design review and optional board bring-up

1. Connected components may disagree on protocols, reset, timing or ownership despite working in isolation.

2. Pin assignment, voltage compatibility, interface requirements and the reviewed board safety plan.

3. No. ASIC physical design, verification, manufacturing and signoff impose additional requirements.

Module 36: Embedded platforms, drivers and board design

36.1 Serial interfaces and peripheral contracts

1. They may disagree about clock polarity, phase, bit order or other interface details.

2. No. Electrical levels and tolerances must be checked separately.

3. A complete tested contract is more useful than several incomplete drivers and makes protocol reasoning manageable.

36.2 Firmware, RTOS scheduling and driver lifecycle

1. No. Task execution, blocking, scheduling assumptions and resource limits must support the deadlines.

2. That can make the watchdog interpret a failed system as healthy.

3. They need a defined completion, cancellation or failure path that preserves ownership and memory safety.

36.3 Schematics, PCB layout, power integrity and signal integrity

1. No. DRC checks chosen geometric rules, not every circuit, component or system assumption.

2. Current loops and reference continuity affect signal quality, coupling and electromagnetic behavior.

3. A visually similar package can have different pin numbering, dimensions or orientation.

36.4 Bring-up, instrumentation and fault isolation

1. Failure in those prerequisites can make every higher-level symptom misleading.

2. It turns bring-up into falsifiable checks rather than random changes.

3. Stop the experiment and investigate without continuing to energize or stress the system.

Module 37: GPU microarchitecture and memory systems

37.1 SIMT execution, divergence, occupancy and latency hiding

1. Threads in one execution group require different control-flow paths, potentially executing them under different active masks.

2. No. Memory limits, dependencies, instruction mix and wasted work can still dominate.

3. It can introduce spills or extra instructions even if more work becomes resident.

37.2 Coalescing, local storage, caches and shared-memory banks

1. The efficiency of combining accesses into memory transactions for suitable address patterns.

2. Bank conflicts, synchronization and limited capacity can constrain it.

3. The application pays that cost even if the converted kernel itself is faster.

37.3 Raster, texture, matrix and ray-tracing units

1. No. Shading, memory traffic, acceleration-structure work and presentation remain.

2. Different precision can change both speed and acceptable output, making raw throughput comparisons misleading.

3. Setup, packing, transfer or scheduling overhead can exceed the saved arithmetic time.

37.4 CPU-GPU sharing, discrete memory and synchronization

1. No. Caches, bandwidth, migration and contention still matter.

2. It can violate ownership and synchronization requirements, creating inconsistent or undefined results.

3. Relevant setup, transfers or migration, computation, synchronization and result access.

Module 38: Quantitative performance engineering

38.1 Amdahl, roofline and bottleneck models

1. 200 GFLOP/s under the stated simplified ceilings and intensity.

2. 1/0.2 = 5.

3. No. Other bottlenecks or inefficiencies may prevent reaching the upper bound.

38.2 Profilers, counters and trustworthy benchmark design

1. To ensure the measured program actually performs the intended correct computation.

2. No. Definitions and collection behavior can differ.

3. It reduces selective removal of inconvenient samples and makes the analysis auditable.

38.3 SIMD, cache blocking, NUMA and job scheduling

1. Independent logical data share a coherence unit, causing traffic when different cores modify it.

2. Scheduling and synchronization overhead can exceed useful work.

3. The optimization may lose when setup and scheduling costs dominate.

38.4 Energy, thermal behavior and sustained performance

1. 100 joules.

2. 80 W for 1 second uses 80 J, less than 100 J, under the simplified model.

3. That can damage equipment or create unsafe conditions and invalidates normal-use performance claims.

Module 39: Gaming-platform constraints and interactive latency

39.1 Input-to-display latency and frame pacing

1. No. Queueing and other stages can change response delay at the same frame rate.

2. Approximately 33.3 ms.

3. Different timing points include different portions of the interaction path.

39.2 Mobile and handheld systems: memory, power and lifecycle

1. The operating environment can impose strict lifecycle limits or terminate unexpectedly.

2. Thermal and power behavior can change available performance over time.

3. Correct game rules, essential information and a coherent user experience.

39.3 XR spaces, stereo views, pose timing and comfort constraints

1. Its coordinates and relevance to presentation depend on both.

2. No. Use the runtime-provided view and projection configuration.

3. No. Prediction has errors and only addresses parts of the overall system.

39.4 Portability, platform services and console-development boundaries

1. It reveals assumptions that the original permissive environment accidentally allowed.

2. No. Access and use depend on the platform's authorization and agreements.

3. They have different mutability, lifecycle and platform storage requirements.

Module 40: Hardware/software co-design capstone

40.1 Workload characterization and measurable requirements

1. They establish whether the proposed hardware addresses a real limitation and what it must support.

2. 0.2/20 = 1%.

3. Yes, when evidence shows a simpler or cheaper alternative satisfies the requirements.

40.2 Architecture alternatives, arithmetic precision and dataflow

1. Memory supply, dependencies or other resources may remain the bottleneck.

2. The application's correctness or quality requirements, not only hardware convenience.

3. It exposes tradeoffs and tests whether the selected design is actually preferable under the requirements.

40.3 Interface design, virtualization and end-to-end evaluation

1. They control memory access and resource use and can otherwise violate bounds or isolation.

2. Yes. The application includes overhead and unaccelerated work outside the kernel.

3. Setup amortization and latency can change dramatically with batch size.

40.4 Design review, evidence tiers and undergraduate CE completion

1. They support different levels of confidence and have different modeling limitations.

2. It is correct, reasonably configured and optimized enough that the comparison is not won through avoidable mistakes.

3. No. It demonstrates educational design competence, not the full professional signoff and manufacturing process.

Module 41: Graduate numerical methods and research practice

41.1 Reading research as claims, assumptions and evidence

1. They control different quantities and can support different performance conclusions.

2. It connects each conclusion to its actual mathematical or experimental support and exposes unsupported generalization.

3. No. Configuration, implementation, data, numerical and interpretation differences must first be investigated.

41.2 Conditioning, sparse systems and iterative solvers

1. 10^6.

2. An ill-conditioned system can amplify small equation perturbations into large changes in the solution.

3. A symmetric positive-definite matrix, or the corresponding Hermitian positive-definite complex case.

41.3 Constrained optimization, derivatives and optimality conditions

1. x=y=0.5, with objective 0.5 and multiplier -1 under the stated Lagrangian sign convention.

2. No. It may also be a saddle or maximum.

3. Large steps incur truncation error while extremely small steps amplify floating-point roundoff.

41.4 Experimental design, uncertainty and reproducible graduate work

1. It determines what observations can reasonably be treated as independent for the intended inference.

2. Whether removing or changing a component changes the outcome in a way that supports its claimed contribution.

3. They test whether tuning improvements generalize beyond the cases used to choose parameters.

Module 42: Advanced parallel and GPU algorithms

42.1 Parallel reductions, scans and work-span analysis

1. 10.

2. 1,023.

3. Changing grouping changes rounding because floating-point addition is not exactly associative.

42.2 Tiling, data reuse and asynchronous pipelines

1. They can increase resource usage, reduce residency or cause spills and boundary overhead.

2. A buffer must not be consumed before preparation completes or overwritten before its current consumer finishes.

3. It provides a simpler correctness and portability reference for evaluating the extra pipeline complexity.

42.3 Irregular workloads, wavefront scheduling and load balance

1. A few long tasks can dominate the final tail after most workers become idle.

2. Unschedulable groups may never run while resident groups wait, causing deadlock.

3. No. Sorting cost, added latency and data movement can exceed coherence gains.

42.4 Portable performance and integration-quality evaluation

1. No. Valid specialization or bounded tuning may be necessary across targets.

2. The evidence is limited to that tested configuration; broader portability remains unverified.

3. A standalone kernel gain may not survive real data preparation, scheduling and result consumption.

Module 43: Advanced rendering and reconstruction

43.1 Monte Carlo light transport and multiple importance sampling

1. 0.25 under equal sample counts.

2. Otherwise nonzero contributions can never be sampled and the estimator can be biased or undefined.

3. Approximately four times as many samples.

43.2 Real-time ray tracing, dynamic acceleration and hybrid pipelines

1. Large motion can degrade bounds and increase traversal cost.

2. Not under that stated per-frame workload; total time increases by 3 ms.

3. Dynamic updates and temporal behavior can dominate costs or artifacts absent in a static scene.

43.3 Spatiotemporal reuse, reservoirs and denoising

1. Reuse can introduce correlation and change the estimator's sampling structure.

2. Different surfaces can occupy that pixel after motion or disocclusion.

3. Quality, bias/artifacts, temporal behavior and total runtime, not only the raw ray count.

43.4 Differentiable and neural rendering under engineering constraints

1. No. The modeled computation, discontinuities and stochastic estimator must still match the intended objective.

2. It tests generalization beyond correlated frames or content seen during training.

3. When the claimed practical benefit depends on total preparation, deployment or repeated use, not only one isolated inference.

Module 44: Advanced engine systems and simulation research

44.1 Large-world scheduling, streaming and critical-path control

1. It changes the needed content abruptly, bypassing assumptions about gradual motion.

2. Physics, rendering, networking and stored references must agree on coordinate changes.

3. It limits work and resource use while explicitly choosing which quality or availability guarantees may temporarily relax.

44.2 Deformable simulation, variational models and solver tradeoffs

1. -k*(x-rest).

2. No. A stable trajectory can still have large discretization error or use an unsuitable physical model.

3. They affect different sources of error and can reveal hidden parameter dependence.

44.3 Distributed game state, consistency and overload behavior

1. Two participants may both commit effects that should occur only once under one owner.

2. Many failed or slow requests trigger additional retries that worsen overload.

3. No. It supports only the tested model and scale unless broader evidence is provided.

44.4 Formal specifications, replay and reliability-oriented engine design

1. Safety excludes forbidden events; liveness requires eventual progress under stated assumptions.

2. An environment that never lets work run can prevent completion regardless of the local algorithm.

3. No. The relationship between the model and implementation must be established and checked.

Module 45: Compilers, shader languages and engine specialization

45.1 Intermediate representations, SSA and dataflow analysis

1. One defining assignment in the representation.

2. The else predecessor supplies 2 unless that path is proven unreachable.

3. It marks a state where further propagation under the analysis rules no longer changes the computed facts.

45.2 Alias analysis, vectorization and legal optimization

1. Two apparently different references may access the same storage, creating dependencies.

2. No. Each depends on the previous result.

3. They can permit transformations that change rounding, exceptional-value handling or other required semantics.

45.3 Domain-specific languages, specialization and safe JIT execution

1. 16.

2. Restrictions make validation, optimization and safe execution more tractable.

3. It provides a portable fallback and a simpler reference for semantic testing.

45.4 Compiler/hardware co-design and graduate artifact review

1. Packing, spills, data movement or unaccelerated work can outweigh the arithmetic savings.

2. Reproduction recreates an existing result; extension changes or adds a clearly identified method, setting or analysis.

3. No. The requirement is rigorous, reproducible work and an independent technical review.

Module 46: Advanced computer architecture for gaming workloads

46.1 Cache coherence, memory consistency and synchronization

1. Coherence concerns copies and writes of a location; consistency constrains observable ordering across memory operations and locations.

2. No. Language data-race and synchronization rules still apply.

3. Requests and responses overlap in time, creating states and races absent from a purely stable-state diagram.

46.2 Out-of-order execution, speculation and precise state

1. False name dependencies such as write-after-read and write-after-write, not true read-after-write data dependencies.

2. To preserve the instruction-set contract and support precise state at exceptions or recovery.

3. The thread may be limited by dependencies, cache misses, branch behavior or insufficient instruction-level parallelism.

46.3 Architecture simulation, traces and model validation

1. No. Accuracy depends on the modeled mechanisms, calibration and workload assumptions.

2. It reveals whether conclusions depend strongly on uncertain or selected parameters.

3. Feedback in which timing changes the future execution or interaction pattern that generated the trace.

46.4 Memory systems, chiplets and multi-objective design

1. No. A uses less energy but B has lower latency.

2. Another design is no worse on either metric and strictly better on at least one; here both A and B improve both metrics.

3. Its manufacturing, yield, packaging and commercial assumptions are not verified production terms.

Module 47: VLSI, timing and physical implementation

47.1 CMOS logic, transistor sizing and delay models

1. 5 normalized delay units.

2. The enlarged gate presents greater input capacitance to its driver.

3. No. It needs a justified technology-dependent scale or characterized timing model.

47.2 Static timing analysis, clocks and timing exceptions

1. 0.65 ns before omitted effects and margins.

2. No. It only removes that path from analysis and is valid only when the functional contract justifies it.

3. Setup and hold impose different requirements and can fail at different corners or paths.

47.3 Synthesis, placement, clock trees, routing and physical checks

1. Actual interconnect parasitics and placement/routing choices add delays not captured by simpler estimates.

2. No. Physical implementation and many additional checks remain.

3. They define rules and models that materially affect implementation and reported results.

47.4 Power, area, process corners and design-space closure

1. 0.81, or a 19% reduction in the simplified dynamic component.

2. Switching frequency of nodes strongly affects dynamic power.

3. Transistor behavior and circuit delay change, so the original frequency may no longer meet margins.

Module 48: Verification, reliability and hardware trust boundaries

48.1 Formal properties, abstraction and assume-guarantee reasoning

1. No. It is an unresolved result under that run and resource limit.

2. The required service behavior is never exercised.

3. Each block's environment, including other blocks, actually satisfies its assumptions without circular or inconsistent reasoning.

48.2 Clock/reset crossings and asynchronous FIFO verification

1. Adjacent values change one bit, reducing ambiguity from simultaneous multi-bit transitions under the intended crossing design.

2. Yes, especially when it occurs between destination sampling edges.

3. No. That requires physical device, timing and implementation assumptions not captured by ordinary digital logic alone.

48.3 Fault models, ECC, testability and recovery

1. No. Two flips can preserve parity.

2. The guarantee applies only to the modeled failure types and assumptions.

3. It lacks a defined safe response, so detection alone may not preserve useful or correct operation.

48.4 Hardware security, thermal limits and assurance review

1. Their combination determines the memory range the device may access and can overflow or exceed authorized bounds.

2. Yes. Timing, shared resources or other observable behavior may reveal information.

3. No. It is bounded educational evidence with stated assumptions and residual risks.

Module 49: Master's thesis: proposal, implementation and evaluation

49.1 Question selection, literature map and thesis scope

1. It specifies an observation or result that could count against the proposed claim.

2. They prevent uncontrolled scope growth and make the intended contribution assessable.

3. No. Relevant newer and competing work must be reviewed when the project begins.

49.2 Proposal review, baselines and risk-reduction milestones

1. It establishes feasibility, comparison quality and the measurement path before major new implementation.

2. It produces a reviewable artifact or result rather than only consuming time.

3. It reduces ad hoc decisions that hide failure or endlessly expand the project.

49.3 Implementation, ablations and controlled evaluation

1. No. The original target must be evaluated on its own terms.

2. They define the method's actual reliability and prevent selective reporting.

3. Check implementation and measurement, examine assumptions, and report the conclusion supported by the evidence rather than manufacturing a success story.

49.4 Thesis writing, reproducibility and contribution boundaries

1. A defined baseline, workload, metric, environment, comparison quality and uncertainty or measurement context.

2. The data may support several explanations, and an inferred cause needs its own evidence.

3. The exact new work, reproduced components, scope and remaining unverified claims.

Module 50: Thesis defense, professional practice and continued development

50.1 Oral defense and independent artifact reproduction

1. Whether the learner can reason about, predict and change the actual system.

2. No. A rigorous negative or limited result can pass when the claim and evidence are sound.

3. The author's environment may contain undeclared dependencies or manual steps absent from the artifact.

50.2 Mature-codebase integration, review and maintainable contributions

1. It may violate ordering, lifetime, serialization or platform contracts outside the benchmark.

2. No. A reviewed local contribution with evidence can satisfy the educational objective.

3. It increases risk before the learner understands the codebase's hidden constraints and maintenance needs.

50.3 Portfolio, technical interviews and role-specific evidence

1. Clear scope, ownership, reproducible evidence and stated limitations.

2. No. It is a self-study curriculum unless separately assessed and credited by an accredited institution.

3. They test whether concepts generalize beyond memorized exercises or copied implementations.

50.4 Long-term learning, maintenance and final handoff

1. It preserves the known reproducible baseline while evaluating compatibility and regressions.

2. Architecture, build/test commands, ownership, known risks, data locations and the next decisions.

3. Depth and sustained evidence usually benefit from focused work rather than simultaneous shallow pursuit of every advanced topic.

Entry diagnostic: answer guidance

Match these numbers to the 15-question entry diagnostic in PROGRAM_GUIDE. Use the result to locate missing prerequisites, not as an intelligence test or a reason to skip untested skills.

1. 11; multiplication precedes addition.

2. 250 milliseconds.

3. x=4.

4. 256.

5. (1,4).

6. 5, from sqrt(3^2+4^2).

7. 6.

8. 5. Reassigning x does not recompute y.

9. 1, 2 and 3.

10. 20.

11. It may run forever because its condition never becomes false and no exit occurs.

12. False.

13. 4, 5 and 6 test below, at and above the boundary.

14. The process working directory and the exact filename/path.

15. No. Correctness on the stated supported inputs is required before calling it an acceptable optimization.